

WARP: An Efficient Engine for Multi-Vector Retrieval

WARP: 高效多向量检索引擎

Jan Luca Scheerer*

扬·卢卡·舍雷尔 *

lscheerer@ethz.ch

ETH Zurich

苏黎世联邦理工学院

Switzerland

瑞士

Matei Zaharia

马泰·扎哈里亚

matei@berkeley.edu

UC Berkeley

加州大学伯克利分校

United States

美国

Christopher Potts

克里斯托弗·波茨

cgpotts@stanford.edu

Stanford University

斯坦福大学

United States

美国

Gustavo Alonso

古斯塔沃·阿隆索

alonso@inf.ethz.ch

ETH Zurich

苏黎世联邦理工学院

Switzerland

瑞士

Omar Khattab

奥马尔·哈塔卜

okhattab@cs.stanford.edu

Stanford University

斯坦福大学

United States

美国

Abstract

摘要

We study the efficiency of multi-vector retrieval methods like ColBERT and its recent variant XTR. We introduce WARP, a retrieval engine that drastically improves the efficiency of XTR-based ColBERT retrievers through three key innovations: (1) WARPSELECT for dynamic similarity imputation, (2) implicit decompression during retrieval, and (3) a two-stage reduction process for efficient scoring. Thanks also to highly-optimized C++ kernels and to the adoption of specialized inference runtimes, WARP can reduce end-to-end query latency relative to XTR's reference implementation by 41x, and thereby achieves a 3x speedup over the official ColBERTv2 PLAID engine, while preserving retrieval quality.

我们研究了 ColBERT 及其近期变体 XTR 等多向量检索方法的效率。提出 WARP 检索引擎，通过三项关键创新显著提升基于 XTR 的 ColBERT 检索效率：(1) 动态相似性插补的 WARPSELECT，(2) 检索期间的隐式解压缩，(3) 高效打分的两阶段归约流程。得益于高度优化的 C++ 内核和专用推理运行时的采用，WARP 相较 XTR 参考实现可降低 41x 的端到端查询延迟，在保持检索质量的同时实现比官方 ColBERTv2 PLAID 引擎 3x 的加速。

C <https://github.com/jlscheerer/xtr-warp>

C <https://github.com/jlscheerer/xtr-warp>

Keywords

关键词

Dense Retrieval, Multi-Vector, Late Interaction, Efficiency

稠密检索, 多向量, 延迟交互, 效率

ACM Reference Format:

ACM 参考文献格式:

Jan Luca Scheerer, Matei Zaharia, Christopher Potts, Gustavo Alonso, and Omar Khattab. 2025. WARP: An Efficient Engine for Multi-Vector Retrieval. In Preprint. Preprint, 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnnnn>

Jan Luca Scheerer, Matei Zaharia, Christopher Potts, Gustavo Alonso 与 Omar Khattab。2025。WARP: 高效多向量检索引擎。预印本。9 页。<https://doi.org/10.1145/nnnnnnn.nnnnnnnnn>

1 Introduction

1 引言

Over the past several years, information retrieval (IR) research has introduced new neural paradigms for search based on pretrained Transformers. Central among these, the late interaction paradigm proposed in ColBERT [8] departs the bottlenecks of conventional single-vector representations. It instead encodes queries and documents into multi-vector representations on top of which it is able to scale gracefully to search massive collections.

过去数年, 信息检索 (IR) 研究提出了基于预训练 Transformer 的新型神经搜索范式。其中 ColBERT[8] 提出的延迟交互范式突破了传统单向量表示的瓶颈, 将查询和文档编码为多向量表示, 从而能优雅地扩展至海量集合搜索。

Since the original ColBERT was introduced, there has been substantial research in optimizing the latency of multi-vector retrieval models [2, 12, 13]. Perhaps most notably, PLAID [16] reduces late interaction search latency by $45\times$ on a CPU compared to a vanilla ColBERTv2 [17] process, while continuing to deliver state-of-the-art retrieval quality. Orthogonally, Google DeepMind’s ConteXtualized Token Retriever (XTR) [10] introduces a novel training objective that eliminates the need for a separate gathering stage and thereby significantly simplifies the subsequent scoring stage. While XTR lays extremely promising groundwork for more efficient multi-vector retrieval¹, we find that it relies naively on a general purpose vector similarity search library (ScANN) and combines that with native Python data structures and manual iteration, introducing substantial overhead.

自 ColBERT 问世以来，多向量检索模型的延迟优化研究取得显著进展 [2,12,13]。最引人注目的是 PLAID[16] 在 CPU 上相较原生 ColBERTv2[17] 流程实现 45x 的延迟降低，同时保持顶尖检索质量。Google DeepMind 的语境化标记检索器 (XTR)[10] 则提出新型训练目标，省去独立聚合阶段从而大幅简化后续打分流程。虽然 XTR 为高效多向量检索奠定极佳基础¹，但我们发现其简单依赖通用向量相似性搜索库 (ScANN)，结合原生 Python 数据结构和手动迭代，引入显著开销。

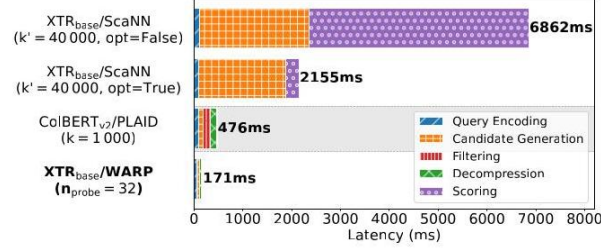


Figure 1: Single-threaded CPU latency breakdown of the unoptimized reference implementation from (1) XTR,² (2) a variant of XTR that we optimized, (3) the official PLAID system, and (4) our proposed WARP on LoTTE Pooled.

图 1:LoTTE Pooled 数据集上 (1)XTR 未优化参考实现、(2) 我们优化的 XTR 变体、(3) 官方 PLAID 系统、(4) 提出的 WARP 在单线程 CPU 的延迟分解。

The key insights from PLAID and XTR appear rather isolated. Whereas PLAID is concerned with aggressively and swiftly pruning away documents it finds unpromising, XTR tries to eliminate gathering complete document representations in the first place. We ask whether there are potential rich interactions between these two fundamentally distinct approaches to speeding up multi-vector search. To study this, we introduce a new engine for retrieval with XTR-based ColBERT models, called WARP, that combines techniques from ColBERTv2/PLAID with innovations tailored for the XTR architecture. Our contributions in WARP include: (1) the WARP method for imputing missing similarities, (2) a new method for implicit decompression of vectors during search, and (3) a novel two-stage reduction phase for efficient scoring.

PLAID 与 XTR 的核心洞见相对孤立:PLAID 侧重快速剪枝低潜力文档，XTR 则致力于避免完整文档表示聚合。我们探究这两种加速多向量搜索的根本不同方法是否存在深度协同可能。为此提出 WARP 引擎，融合 ColBERTv2/PLAID 技术与 XTR 架构创新，贡献包括:(1) 缺失相似性插补的 WARP 方法，(2) 搜索期间向量隐式解压缩新技术，(3) 高效打分的新两阶段归约流程。

Experimental evaluation shows that WARP achieves a 41x reduction in end-to-end latency compared to the XTR reference implementation on LoTTE Pooled, bringing query response times down from above 6 seconds to just 171 milliseconds in single-threaded execution, while also reducing index size by a factor of 2x – 4x compared to the ScaNN-based baseline. Furthermore, WARP demonstrates a 3x speedup over the state-of-the-art ColBERTv2/PLAID system, as illustrated in Figure 1.

实验表明，WARP 在 LoTTE Pooled 上相较 XTR 参考实现实现 41x 的端到端延迟降低，单线程执行将查询响应时间从 6 秒以上降至 171 毫秒，同时索引尺寸较基于 ScaNN 的基线缩小 2x – 4x 倍。如图 1 所示，WARP 相对最先进 ColBERTv2/PLAID 系统还实现 3x 加速。

After briefly reviewing prior work on efficient neural information retrieval in Section 2, we analyze the latency bottlenecks in the ColBERT and XTR retrieval frameworks in Section 3, identifying key areas for optimization within the XTR framework. These findings form the foundation for our work on WARP, which we introduce and describe in detail in Section 4. In Section 5, we evaluate WARP’s end-to-end latency and scalability using the BEIR [18] and LoTTE [17] benchmarks. Finally, we compare our implementation to existing state-of-the-art engines.

在第二节简要回顾了高效神经信息检索的先前工作后，我们在第三节分析了 ColBERT 和 XTR 检索框架的延迟瓶颈，确定了 XTR 框架内优化的关键领域。这些发现构成了我们在第四节详细介绍的 WARP 研究基础。第五节通过 BEIR[18] 和 LoTTE[17] 基准测试评估了 WARP 的端到端延迟和可扩展性。最后，我们将实现方案与现有最先进引擎进行对比。

2 Related Work

2 相关工作

Dense retrieval models can be broadly categorized into single-vector and multi-vector approaches. Single-vector methods, exemplified by ANCE [19] and STAR/ADORE [20], encode a passage into a single dense vector [7]. While these techniques offer computational efficiency, their inherent limitation of representing complex documents with a single vector has been shown to constrain the model’s ability to capture intricate information structures [8].

稠密检索模型可大致分为单向量和多向量方法。以 ANCE[19] 和 STAR/ADORE[20] 为代表的单向量方法将文本编码为单个稠密向量 [7]。虽然这些技术具有计算效率优势，但用单一向量表示复杂文档的固有局限已被证明会制约模型捕捉精细信息结构的能力 [8]。

To address such limitations, ColBERT [8] introduces a multi-vector paradigm. In this approach, both queries and documents are independently encoded as multiple embeddings, allowing for a richer representation of document content and query intent. The multi-vector approach is further refined in ColBERTv2 [17], which improves supervision and incorporates residual compression to significantly reduce the space requirements associated with storing multiple vectors per indexed document. Building upon these innovations, PLAID [16] substantially accelerates ColBERTv2 by efficient pruning non-relevant passages using the residual representation and by employing optimized C++ kernels. EMVB [13] further optimizes PLAID’s memory usage and single-threaded query latency using product quantization [6] and SIMD instructions.

*Work completed as a visiting student researcher at Stanford University.

* 斯坦福大学访问学生研究员期间完成的工作

¹ <https://github.com/google-deepmind/xtr>

¹ <https://github.com/google-deepmind/xtr>

针对这些局限, ColBERT[8] 提出了多向量范式。该方法将查询和文档独立编码为多个嵌入向量, 从而更丰富地表征文档内容和查询意图。ColBERTv2[17] 通过改进监督机制和引入残差压缩技术, 大幅降低了每个索引文档存储多向量的空间需求。基于这些创新, PLAID[16] 利用残差表示高效剪除非相关段落, 并采用优化的 C++ 内核, 显著加速了 ColBERTv2。EMVB[13] 则通过乘积量化 [6] 和 SIMD 指令进一步优化了 PLAID 的内存使用和单线程查询延迟。

Separately, COIL [2] incorporates insights from conventional retrieval systems [15] by constraining token interactions to lexical matches between queries and documents. SPLATE [1] translates the embeddings produced by ColBERTv2 style pipelines to a sparse vocabulary, allowing the candidate generation step to be performed using traditional sparse retrieval techniques. CITADEL [12] introduces conditional token interaction through dynamic lexical routing, selectively considering tokens for relevance estimation. While CITADEL significantly reduces GPU execution time, it falls short of PLAID’s CPU performance at comparable retrieval quality.

另一方面, COIL[2] 借鉴传统检索系统 [15] 的思想, 将词元交互限制在查询与文档的词汇匹配层面。SPLATE[1] 将 ColBERTv2 风格流程生成的嵌入向量转换为稀疏词汇表示, 使得候选生成步骤可采用传统稀疏检索技术完成。CITADEL[12] 通过动态词汇路由实现条件式词元交互, 选择性评估词元相关性。虽然 CITADEL 大幅降低了 GPU 执行时间, 但在同等检索质量下仍不及 PLAID 的 CPU 性能。

The Contextualized Token Retriever (XTR) [10], introduced by Lee et al. [10], represents a notable conceptual advancement in dense retrieval. XTR simplifies the scoring process and eliminates the gathering stage entirely, theoretically enhancing retrieval efficiency. However, its current end-to-end latency limits its application in production environments where query response time is critical or GPU resources are constrained.

Lee 等人 [10] 提出的上下文感知词元检索器 (XTR)[10] 代表了稠密检索领域的重要概念突破。XTR 简化了评分流程并完全取消了聚集阶段, 理论上提升了检索效率。然而其当前的端到端延迟制约了在查询响应时间敏感或 GPU 资源受限的生产环境中的应用。

3 Latency of Current Neural Retrievers

3 当前神经检索器的延迟分析

We start by analyzing two state-of-the-art multi-vector retrieval methods to identify their bottlenecks, providing the foundation for our work on WARP. We evaluate the latency of PLAID and XTR across various configurations and datasets: BEIR NFCorpus, LoTTE Lifestyle, and LoTTE Pooled. In XTR, token retrieval emerges as a fundamental bottleneck: the need to retrieve a large number of candidates from the ANN backend significantly impacts performance. PLAID, while generally far more efficient, faces challenges in its decompression stage. Query encoding emerges as a shared limitation for both engines, particularly on smaller datasets. These insights inform the design of WARP, which we introduce in Section 4.

我们首先分析两种最先进的多向量检索方法以识别其瓶颈，为 WARP 研究奠定基础。我们在 BEIR NFCorpus、LoTTE Lifestyle 和 LoTTE Pooled 数据集上评估了 PLAID 和 XTR 在不同配置下的延迟。XTR 中，词元检索成为基础性瓶颈：需要从 ANN 后端检索大量候选者显著影响性能。PLAID 虽然整体效率更高，但其解压阶段面临挑战。查询编码是两种引擎共有的限制因素，在小型数据集上尤为明显。这些发现指导了第四节介绍的 WARP 设计。

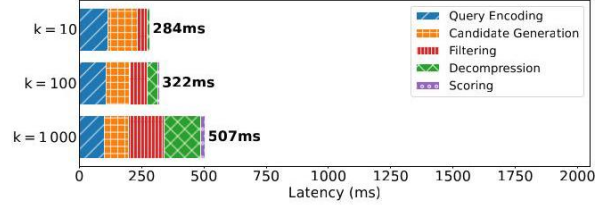


Figure 2: Breakdown of ColBERTv2/PLAID’s avg. latency for varying k on LoTTE Pooled (Dev.)

图 2: ColBERTv2/PLAID 在 LoTTE Pooled(开发集) 上随 k 变化的平均延迟分解

3.1 ColBERTv2/PLAID

3.1 ColBERTv2/PLAID

As shown in Figure 2, we evaluate PLAID’s performance using its optimized implementation [4] and the ColBERTv2 checkpoint from Hugging Face [3]. We configure PLAID’s hyperparameters similar to the original paper [16]. Consistent with prior work [13], we observe single threaded CPU exceeding 500ms on LoTTE Pooled. Furthermore, we find that the decompression stage remains rather constant for fixed k across all datasets, consuming approximately 150 – 200 ms for $k = 1000$. Notably, for smaller datasets like NFCorpus and large k values, this stage constitutes a significant portion of the overall query latency. Thus, the decompression stage emerges as a critical bottleneck for small datasets. Query encoding contributes significantly to overall latency, particularly for smaller datasets and candidate generation constitutes a fixed cost based on the number of centroids. As anticipated, the filtering stage’s execution time is proportional to the number of candidates, increasing for larger k values and bigger datasets. Interestingly, the scoring stage appears to have a negligible impact on ColBERTv2/PLAID’s overall latency across all measurements.

如图 2 所示，我们使用优化实现 [4] 和 Hugging Face[3] 的 ColBERTv2 检查点评估 PLAID 性能。PLAID 超参数配置与原始论文 [16] 保持一致。与先前研究 [13] 一致，我们观察到在 LoTTE Pooled 数据集上单线程 CPU 延迟超过 500ms。此外发现解压阶段耗时在固定 k 条件下跨数据集保持稳定，约消耗 150 – 200 ms (当 $k = 1000$ 时)。值得注意的是，在 NFCorpus 等小型数据集和大 k 值情况下，该阶段占总查询延迟的很大比重。因此解压阶段成为小型数据集的关键瓶颈。查询编码对总体延迟贡献显著，特别是在小型数据集上，而候选生成则基于中心点数量产生固定成本。符合预期的是，过滤阶段执行时间与候选数量成正比，随 k 值和数据集规模增大而增加。值得注意的是，在所有测量中评分阶段对 ColBERTv2/PLAID 总体延迟的影响微乎其微。

3.2 XTR/ScaNN

3.2 XTR/ScaNN

To enable benchmarking of the XTR framework, we developed a Python library based on Google DeepMind’s published code [11]. The library’s code, along with scripts to reproduce the benchmarks, will be made available on GitHub. Unless otherwise specified, all benchmarks utilize the XTR BASE_EN transformer model for encoding. This model was published and is available on Hugging Face [9]. In accordance with the paper [10], we evaluate the implementation for $k' = 1000$ and $k' = 40000$.

为实现 XTR 框架的基准测试，我们基于 Google DeepMind 公布的代码 [11] 开发了 Python 库。该库代码及复现基准测试的脚本将在 GitHub 开源。除非特别说明，所有基准测试均采用 Hugging Face 平台 [9] 发布的 XTR BASE_EN 变压器模型进行编码。根据论文 [10] 所述，我们针对 $k' = 1000$ 和 $k' = 40000$ 场景评估了该实现方案。

As our measurements show, the scoring stage constitutes a significant bottleneck in the end-to-end latency of the XTR framework, particularly when dealing with large k' values, as seen in Figure 3. We argue that this performance bottleneck is largely attributed to an unoptimized implementation in the released code, which relies on native Python data structures and manual iteration, introducing substantial overhead, especially for large numbers of token embeddings. We refactored this implementation to leverage optimized data structures and vectorized operations. This helps us uncover hidden performance inefficiencies and establish a baseline for further optimization, with our improved implementation of XTR to be made publicly available.

测量数据显示，评分阶段构成 XTR 框架端到端延迟的主要瓶颈，这在处理较大 k' 值时尤为明显（见图 3）。我们认为该性能瓶颈主要源于发布代码中未优化的实现方案——其依赖原生 Python 数据结构与手动迭代操作，尤其在处理大量词嵌入时会产生显著开销。我们通过重构实现方案，采用优化数据结构和向量化运算，从而揭示潜在性能缺陷并建立进一步优化的基准，改进后的 XTR 实现将公开发布。

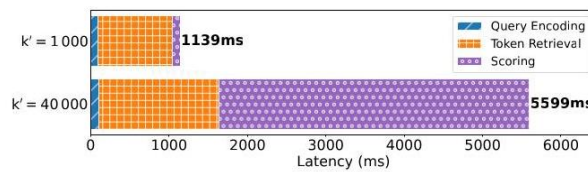


Figure 3: Breakdown of the Google DeepMind’s reference implementation of XTR_{base} /ScaNN’s avg. latency for varying k on LoTTE Pooled [17]

图 3: Google DeepMind 参考实现中 XTR_{base} /ScaNN 在 LoTTE Pooled[17] 数据集上随 k 变化的平均延迟分解

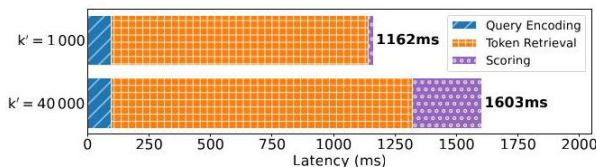


Figure 4: Figure 4 (c): Breakdown of $XTR_{base} / ScaNN$'s avg. latency for varying k' on LoTTE Pooled [17]

图 4(c): $XTR_{base} / ScaNN$ 在 LoTTE Pooled[17] 数据集上随 k' 变化的平均延迟分解

We present the evaluation of our optimized implementation in Figure 4. Notably, the optimized implementation's end-to-end latency is significantly lower than that of the reference implementation ranging from an end-to-end 3.5x speed-up on LoTTE pooled to a 6.3x speed-up on LoTTE Lifestyle for $k = 1000$. This latency reduction is owed in large parts to a more efficient scoring implementation - 14x speed-up on LoTTE Pooled, see ???. In particular, this reveals token retrieval as the fundamental bottlenecks of the XTR framework. Furthermore, we notice that query encoding constitutes a large fraction of the overall end-to-end CPU latency on smaller datasets. While the optimized scoring stage constitutes a small fraction of the overall end-to-end latency, it is still slow in absolute terms - ranging from 33 ms to 281 ms for $k = 1000$ on BEIR NFCorpus and LoTTE Pooled, respectively.

图 4 展示了优化实现的评估结果。值得注意的是，优化方案的端到端延迟显著低于参考实现：在 LoTTE pooled 数据集上实现 3.5 倍加速，在 LoTTE Lifestyle 数据集 $k = 1000$ 场景下达到 6.3x 倍加速。这种延迟降低主要得益于更高效的评分实现（LoTTE Pooled 上 14 倍加速，参见??）。分析表明词条检索是 XTR 框架的根本瓶颈。此外我们发现，在较小数据集上，查询编码占整体端到端 CPU 延迟的主要部分。尽管优化后的评分阶段占比较小，但绝对速度仍较慢——BEIR NFCorpus 和 LoTTE Pooled 数据集上 $k = 1000$ 场景分别需要 33 ms 至 281 ms。

4 WARP

4 WARP 架构

WARP optimizes retrieval for the refined late interaction architecture introduced in XTR. Seeking to find the best of the XTR and PLAID worlds, WARP introduces the novel WARPseLECT algorithm for candidate generation, which effectively avoids gathering token-level representations, and proposes an optimized two-stage reduction for faster scoring via a dedicated C++ kernel combined with implicit decomposition. WARP also uses specialized inference runtimes for faster query encoding.

WARP 针对 XTR 提出的精细化延迟交互架构优化检索流程。为融合 XTR 与 PLAID 的优势，WARP 创新性地提出 WARPseLECT 候选生成算法，有效避免收集词级表征，并通过专用 C++ 内核结合隐式解压技术实现优化的两阶段降维加速评分。该架构还采用专用推理运行时加速查询编码。

As in XTR, queries and documents are encoded independently into embeddings at the token-level using a fine-tuned T5 transformer [14]. To scale to large datasets, document representations are computed in advance and constitute WARP's index. The similarity between a query q and document d is modeled using the XTR's adaptation of ColBERT's summation of MaxSim operations Equation (1):³

与 XTR 相同，查询和文档通过微调 T5 变压器 [14] 在词级独立编码为嵌入向量。为支持大规模数据集，文档表征被预先计算并构成 WARP 索引。查询 q 与文档 d 的相似度采用 XTR 改进的 ColBERT 最大相似度求和公式 (1) 建模：³

$$S_{d,q} = \sum_{i=1}^n \max_{1 \leq j \leq m} [\hat{\mathbf{A}}_{i,j} q_i^T d_j + (1 - \hat{\mathbf{A}}_{i,j}) m_i] \quad (1)$$

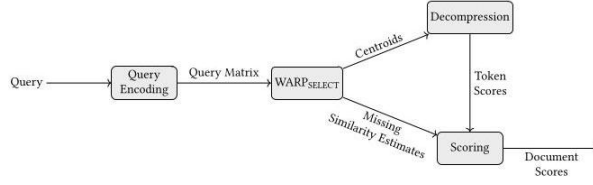


Figure 5: WARP Retrieval consisting of query encoding, WARP_{SELECT}, decompression, and scoring. Notably, centroid selection is combined with the computation of missing similarity estimates in WARPseLECT

图 5: WARP 检索流程包含查询编码、WARP_{SELECT}、解压和评分阶段。值得注意的是, WARPseLECT 将质心选择与缺失相似度估计计算进行了融合

where q and d are the matrix representations of the query and passage embeddings respectively, m_i denotes the missing similarity estimate for q_i , and $\hat{\mathbf{A}}$ describes XTR’s alignment matrix. In particular, $\hat{\mathbf{A}}_{i,j} = \mathbb{1}_{[j \in \text{top-}k'_j(\mathbf{d}_{i,j'})]}$ captures whether a document token embedding of a candidate passage was retrieved for a specific query token q_i as part of the token retrieval stage. We refer to [8, 10] for an intuition behind this choice of scoring function.

其中 q 和 d 分别表示查询向量与段落向量的矩阵表示, m_i 表示 q_i 的缺失相似度估计值, $\hat{\mathbf{A}}$ 则描述 XTR 对齐矩阵。具体而言, $\hat{\mathbf{A}}_{i,j} = \mathbb{1}_{[j \in \text{top-}k'_j(\mathbf{d}_{i,j'})]}$ 记录了候选段落的文档词元向量是否在词元检索阶段被检索用于特定查询词元 q_i 。关于该评分函数选择背后的逻辑, 请参阅 [8, 10]。

4.1 Index Construction

4.1 索引构建

Akin to ColBERTv2 [17], WARP’s compression strategy involves applying k -means clustering to the produced document token embeddings. As in ColBERTv2, we find that using a sample of all passages proportional to the square root of the collection size to generate this clustering performs well in practice. After having clustered the sample of passages, all token-level vectors are encoded and stored as quantized residual vectors to their nearest cluster centroid. Each dimension of the quantized residual vector is a b -bit encoding of the delta between the centroid and the original uncompressed vector. In particular, these deltas are stored as a sequence of $128 \cdot \frac{b}{8}$ 8-bit values, wherein 128 represents the transformers token embedding dimension. Typically, we set $b = 4$, i.e., compress each dimension of the residual into a single nibble, for an $8x$ compression⁴. In this case, each compressed 8-bit value stores 2 indices in the range $[0, 2^b]$. Instead of quantizing the residuals uniformly, WARP uses quantiles derived from the empirical distribution to determine bucket boundaries (bucket_cutoffs) and the corresponding representative values (bucket_weights). This process allows WARP to allocate more quantization levels to densely populated regions of the data distribution, thereby minimizing the overall quantization error for residual compression.

与 ColBERTv2[17] 类似, WARP 的压缩策略对生成的文档词元向量应用 k -均值聚类。实践表明, 采用与集合规模平方根成正比的段落样本进行聚类效果良好。完成样本聚类后, 所有词元级向量均被编码存储为最近聚类中心的量化残差向量。量化残差向量的每个维度都是中心点与原始未压缩向量间差值的 b 位编码。这些差值以 $128 \cdot \frac{b}{8}$ 个 8 位值序列存储 (128 表示 Transformer 词元嵌入维度)。通常我们设定 $b = 4$, 即将残差向量的每个维度压缩为半字节, 实现 $8x$ 倍压缩率⁴。此时每个压缩后的 8 位值可存储 $[0, 2^b]$ 范围内的 2 个索引。WARP 采用基于经验分布的分位数确定桶边界 (bucket_cutoffs) 及代表值 (bucket_weights), 而非均匀量化残差。该方法能为数据分布密集区域分配更多量化层级, 从而最小化残差压缩的整体量化误差。

4.2 Retrieval

4.2 检索

Extending PLAID, the retrieval process in the WARP engine is divided into four distinct steps: query encoding, candidate generation, decompression, and scoring. Figure 5 illustrates the retrieval process in WARP. The process starts when the query text is encoded into q , a (query_maxlen, 128)-dimensional tensor, using the Transformer model.⁵ The most similar n_{probe} centroids are identified for each of the query_maxlen query token embeddings.

WARP 引擎在 PLAID 基础上将检索流程扩展为四个阶段: 查询编码、候选生成、解压和评分。如图 5 所示, 系统首先通过 Transformer 模型将查询文本编码为 (query_maxlen, 128) 维张量 q 。⁵ 随后为每个 query_maxlen 长度的查询词元向量识别最相似的 n_{probe} 个聚类中心。

Subsequently, WARP identifies all document token embeddings belonging to the clusters of the selected centroids and computes their individual relevance score. Computing this score involves decompressing residuals of these identified document token embeddings and calculating the cosine similarity with the relevant query token embedding. Finally, WARP (implicitly) constructs a $n_{\text{candidates}} \times \text{query_maxlen}$ score matrix S ,⁶ wherein each entry S_{d_i, q_j} contains the maximum retrieved score for the i -th candidate passage d_i and the j -th query token embedding q_j :

接着 WARP 识别属于选定中心点的所有文档词元向量, 计算其个体相关性分数。该过程需解压这些文档词元向量的残差, 并计算与对应查询词元向量的余弦相似度。最终 WARP(隐式) 构建 $n_{\text{candidates}} \times \text{query_maxlen}$ 维评分矩阵 S ,⁶, 其中每个条目 S_{d_i, q_j} 存储第 i 个候选段落 d_i 与第 j 个查询词元向量 q_j 的最高检索得分:

³ Note that we omit the normalization via $\frac{1}{Z}$, as we are only interested in the relative ranking between documents and the normalization constant is identical for any retrieved document.

³ 注意我们省略了 $\frac{1}{Z}$ 标准化步骤, 因仅关注文档间相对排序且标准化常数对所有检索文档相同。

⁴ As compared to an uncompressed 32-bit floating point number per dimension.

⁴ 相较于每维度未压缩的 32 位浮点数。

$$\max_{1 \leq j \leq m} \hat{\mathbf{A}}_{i,j} q_i^T d_j$$

Matrix entries not populated during token retrieval, i.e., $\hat{\mathbf{A}}_{i,j} = 0$, are each imputed with a missing similarity estimate, as postulated in the XTR framework. To compute the relevance score of a document d_i , the cumulative score over all query tokens is computed: $\sum_j S_{d_i, q_j}$. To produce the ordered set of passages, the set of scores sorted and the top k highest scoring passages are returned.

词元检索阶段未填充的矩阵条目 (即 $\hat{\mathbf{A}}_{i,j} = 0$) 均按 XTR 框架假设填充缺失相似度估计值。计算文档 d_i 的相关性分数时, 需汇总所有查询词元的累计得分: $\sum_j S_{d_i, q_j}$ 。最终系统对分数集排序并返回得分最高的 k 个段落。

4.3 WARPselect

4.3 WARPselect

In contrast to ColBERT, which populates the entire score matrix for the items retrieved, XTR only populates the score matrix with scores computed as part of the token retrieval stage. To account for the contribution of any missing tokens, XTR relies on missing similarity imputation, in which they set any missing similarity of the query token for a specific document as the lowest score obtained as part of the gathering stage. The authors argue that this approach is justified as it constitutes a natural upper bound for the true relevance score. In the case of WARP, this bound is no longer guaranteed to hold.⁷

与 ColBERT 为检索项填充完整评分矩阵不同, XTR 仅利用词元检索阶段计算的分数填充矩阵。为补偿缺失词元的影响, XTR 采用缺失相似度估算技术——将查询词元在特定文档中的缺失相似度设为收集阶段获得的最低分。研究者认为该方法合理, 因其构成真实相关性得分的自然上界。但在 WARP 中, 这一界限不再必然成立。⁷

Instead, WARP defines a novel strategy for candidate generation based on cumulative cluster sizes, WARP_{SELECT}. Given the query embedding matrix q and the list of centroids C (Section 4.1), WARP computes the token-level query-centroid relevance scores. As both the query embedding vectors and the set of centroids are normalized, the cosine similarity scores $S_{c,q}$ can be computed efficiently as a matrix multiplication:

WARP 创新性地采用基于累积聚类规模的候选生成策略 WARP_{SELECT}。给定查询嵌入矩阵 q 和质心列表 C (见 4.1 节), WARP 计算词元级查询-质心相关性得分。由于查询嵌入向量和质心集均已归一化, 余弦相似度 $S_{c,q}$ 可通过矩阵乘法高效计算:

$$S_{c,q} = C \cdot q^T$$

Once these relevance scores have been computed WARP identifies the n_{probe} centroids with the largest similarity scores for decompression, as part of candidate generation.

计算完相关性得分后, WARP 选取相似度最高的 n_{probe} 个质心进行解压缩, 作为候选生成环节。

Using these query-centroid similarity scores, WARPseLECT folds the estimation of missing similarity scores into candidate generation. Specifically, for each query token q_i , it sets m_i from Equation (1) as the first element in the sorted list of centroid scores for which the cumulative cluster size exceeds a threshold t' . This method is particularly attractive as all the centroid scores have already been computed and sufficiently sorted as part of candidate generation, so the cost of computing missing similarity imputation with this method is negligible.

WARP 将缺失相似度估算融入候选生成阶段: 对每个查询词元 q_i , 当某质心得分的累积聚类规模超过阈值 t' 时, 将公式 (1) 中的 m_i 设为该质心排序列表的首项。此方法优势在于所有质心得分已在候选生成阶段计算并排序, 因此估算缺失相似度的计算成本可忽略不计。

We find that t' is easy to configure (Section 4.6) without compromising the retrieval quality or efficiency. Unlike XTR, the missing similarity estimate of WARP is inherently tied to the number of retrieved tokens. Intuitively, increasing k' may only help refine the missing similarity estimate, but not significantly increase the density of the score matrix.⁸

我们发现 t' 参数易于配置 (见 4.6 节), 且不影响检索质量或效率。与 XTR 不同, WARP 的缺失相似度估算与检索词元数量内在关联。直观上, 增加 k' 仅能优化缺失相似度估算, 而不会显著提升评分矩阵密度。⁸

4.4 Decompression

4.4 解压缩

The input for the decompression phase is the set of n_{probe} centroid indices for each of the query_maxlen query tokens. Its goal is to calculate relevance scores between each query token and the em-beddings within the identified clusters. For a query token q_i , let $c_{i,j}$, where $j \in [n_{\text{probe}}]$, be the set of centroid indices identified during candidate generation. Let $r_{i,j,k}$ be the set of residuals associated with cluster $c_{i,j}$. The decompression step computes:

解压缩阶段的输入是每个 query_maxlen 查询词元对应的 n_{probe} 个质心索引集合, 其目标是计算查询词元与选定聚类内嵌入向量的相关性得分。对于查询词元 q_i , 令 $c_{i,j}$ (其中 $j \in [n_{\text{probe}}]$) 表示候选生成阶段确定的质心索引集, $r_{i,j,k}$ 表示聚类 $c_{i,j}$ 关联的残差集, 解压缩步骤计算:

$$s_{i,j,k} = \text{decompress}(C[c_{i,j}], r_{i,j,k}) \times q_i^T \forall i, j, k \quad (2)$$

The decompress function converts residuals from their compact form from ColBERTv2 and PLAID into uncompressed vectors. Each residual $r_{i,j,k}$ is composed of 128 indices, each b bits wide. These indices reference values in the bucket weights vector $\omega \in \mathbb{R}^{2^b}$ and are used to offset the centroid $C[c_{i,j}]$. The decompress function is defined as:

解压缩函数将 ColBERTv2 和 PLAID 中的压缩残差转换为原始向量。每个残差 $r_{i,j,k}$ 由 128 个 b 位宽的索引组成, 这些索引指向桶权重向量 $\omega \in \mathbb{R}^{2^b}$ 中的值, 用于调整质心 $C[c_{i,j}]$ 偏移量。解压缩函数定义为:

$$\text{decompress}(C[c_{i,j}], r_{i,j,k}) = C[c_{i,j}] + \sum_{d=1}^{128} \mathbf{e}_d \cdot \omega[(r_{i,j,k})_d] \quad (3)$$

Here, $\mathbf{e}_d$ is the unit vector for dimension d , and $\omega[(r_{i,j,k})_d]$ is the weight value at index $(r_{i,j,k})_d$ for dimension d . In other words, the indices are used to look up specific entries in ω for each dimension independently, adjusting the centroid accordingly.

其中 $\mathbf{e}_d$ 表示维度 d 的单位向量, $\omega[(r_{i,j,k})_d]$ 表示维度 d 上索引 $(r_{i,j,k})_d$ 处的权重值。换言之, 这些索引用于独立查找 ω 中特定条目, 相应调整质心。

Instead of explicitly decompressing residuals as in PLAID, WARP leverages the observation that the scoring function decomposes between centroids and residuals. As a result, WARP reuses the query-centroid relevance scores $S_{c,q}$, computed as part of candidate generation. That is, observe that:

不同于 PLAID 显式解压缩残差的做法, WARP 基于评分函数可分解为质心与残差的性质, 直接复用候选生成阶段计算的查询-质心相关性得分 $S_{c,q}$ 。具体表现为:

$$\begin{aligned} s_{i,j,k} &= \text{decompress}(C[c_{i,j}], r_{i,j,k}) \times q_i^T \\ &= (C[c_{i,j}] \times q_i^T) + \left(\sum_{d=1}^{128} \omega[(r_{i,j,k})_d] q_{i,d} \right) \end{aligned} \quad (4)$$

To accelerate decompression, WARP computes $v = \hat{q} \times \hat{\omega}$, wherein $\hat{q} \in \mathbb{R}^{\text{query_maxlen} \times 128 \times 1}$ represents the query matrix that has been unsqueezed along the last dimension, and $\hat{\omega} \in \mathbb{R}^{1 \times 2^b}$ denotes the vector of bucket weights that has been unsqueezed along the first dimension. With these definitions, WARP can decompress and score candidate tokens via:

为加速解压缩, WARP 计算 $v = \hat{q} \times \hat{\omega}$, 其中 $\hat{q} \in \mathbb{R}^{\text{query_maxlen} \times 128 \times 1}$ 表示沿最后一维展开的查询矩阵, $\hat{\omega} \in \mathbb{R}^{1 \times 2^b}$ 代表沿第一维展开的桶权重向量。基于这些定义, WARP 可通过以下方式解压缩并评分候选标记:

$$\begin{aligned} s_{i,j,k} &= S_{c_j,q_i} + \sum_{d=1}^{128} (\omega \cdot q_{i,d}) [(r_{i,j,k})_d] \\ &= S_{c_j,q_i} + \sum_{i=1}^{128} v_{i,d} [(r_{i,j,k})_d] \end{aligned} \quad (5)$$

⁵ We set query_maxlen = 32 in accordance with the XTR paper.

⁵ 我们根据 XTR 论文设置 query_maxlen = 32。

⁶ Note that our optimized implementation does not physically construct this matrix.

⁶ 注意我们优化后的实现并未实际构建此矩阵。

⁶ For XTR baselines, 'candidate generation' refers to the token retrieval stage.

Note that candidate scoring can now be implemented as a simple selective sum. As the bucket weights are shared among centroids and the query-centroid relevance scores have already been computed during candidate generation, WARP can decompress and score arbitrarily many clusters using $O(1)$ multiplications. This refined scoring function is far more efficient than the one outlined in PLAID,⁹ as it never computes the decompressed embeddings explicitly and instead directly emits the resulting candidate scores. We provide an efficient implementation of the selective sum of Equation (4) and realize unpacking of the residual representation using low-complexity bitwise operations as part of WARP's C++ kernel for decompression.

注意候选评分现在可实现为简单选择性求和。由于桶权重在质心间共享且查询-质心相关分数已在候选生成阶段计算，WARP 可通过 $O(1)$ 次乘法解压缩并评分任意数量聚类。该优化评分函数比 PLAID 所述方案更高效，⁹ 因其从不显式计算解压缩嵌入，而是直接输出候选分数。我们实现了公式 (4) 选择性求和的高效实现，并利用低复杂度位运算解包残差表示，作为 WARP 解压缩 C++ 内核的组成部分。

4.5 Scoring

4.5 评分

At the end of the decompression phase, we have $\text{query_maxlen} \times n_{\text{probe}}$ strides of decompressed candidate document token-level scores and their corresponding document identifiers. Scoring combines these scores with the missing similarity estimates, computed during candidate generation, to produce document-level scores. This process corresponds to constructing the score matrix and taking the row-wise sum.

解压缩阶段结束时，我们获得 $\text{query_maxlen} \times n_{\text{probe}}$ 步长的解压缩候选文档标记级分数及其对应文档标识符。评分将这些分数与候选生成阶段计算的缺失相似度估计结合，生成文档级分数。该过程对应构建分数矩阵并执行行向求和。

Explicitly constructing the score matrix, as in the the reference XTR implementation, introduces a significant bottleneck, particularly for large values of n_{probe} . To address this, WARP efficiently aggregates token-level scores using a two-stage reduction process:

如 XTR 参考实现那样显式构建分数矩阵会形成显著瓶颈，尤其对于较大 n_{probe} 值。为此，WARP 采用两阶段归约高效聚合标记级分数：

⁶ 对 XTR 基线而言，“候选生成”指代标记检索阶段。

⁷ Strictly speaking, the bound is also approximate in XTR's case, as ScaNN does not return the exact nearest neighbors in general.

⁷ 严格来说，XTR 中的边界也是近似值，因 ScaNN 通常不返回精确最近邻。

⁸ This is because tokens retrieved with a larger k' are often from new documents and, therefore, do not refine the scores of already retrieved ones.

⁸ 这是因为较大 k' 检索的标记常来自新文档，因此不会优化已检索标记的分数。

- **Token-level reduction** For each query token, reduce the corresponding set of n_{probe} strides using the max operator. This step implicitly fills the score matrix with the maximum per-token score for each document. As a single cluster can contain multiple document token embeddings originating from the same document, WARP performs inner-cluster max-reduction directly during the decompression phase.

- 标记级归约对每个查询标记，使用 max 算子归约对应 n_{probe} 步长集合。此步骤隐式地用各文档每标记最高分填充分数矩阵。由于单个聚类可能包含同文档的多个标记嵌入，WARP 在解压缩阶段直接执行聚类内最大归约。

- **Document-level reduction** Reduce the resulting strides into document-level scores using a sum aggregation. It is essential to handle missing values properly at this stage - any missing per-token score must be replaced by the corresponding missing similarity estimate, ensuring compliance with the XTR scoring function described in Equation (1). This reduction step corresponds to the row-wise summation of the score matrix.

- 文档级归约使用 sum 聚合将结果步长归约为文档级分数。此阶段必须正确处理缺失值——任何缺失的每标记分数需用对应缺失相似度估计替代，确保符合公式 (1) 所述 XTR 评分函数。该归约步骤对应分数矩阵的行向求和。

After performing both reduction phases, the final stride contains the document-level scores and the corresponding identifiers for all candidate documents. To retrieve the result set, we perform heap select to obtain the top- k documents, similar to its use in the candidate generation phase. Formally, we consider a stride S to be an list of key-value pairs:

完成两阶段归约后，最终步长包含所有候选文档的文档级分数及对应标识符。为获取结果集，我们执行堆选择获取前 k 个文档，类似候选生成阶段用法。形式上，我们将步长 S 视为键值对列表：

$$S = \{(k_i, v_i)\}; K(S) = \{k_i \mid (k_i, v_i) \in S\}; V(S) = \{v_i \mid (k_i, v_i) \in S\}$$

Thus, strides implicitly define a partial function $f_S : K \rightarrow V(S)$:

因此，步长隐式定义部分函数 $f_S : K \rightarrow V(S)$ ：

$$f_S(k) = \begin{cases} v_i & \text{if } \exists v_i. (k, v_i) \in S \\ \perp & \text{otherwise} \end{cases}$$

We define a reduction as a combination of two strides S_1 and S_2 using a binary function r into a single stride by applying r to values of matching keys:

我们将归约定义为使用二元函数 r 将两个步长 S_1 和 S_2 组合为单一步长，通过对匹配键值应用 r 实现：

$$\text{reduce}(r, S_1, S_2) = \{(k, r(f_{S_1}(k), f_{S_2}(k))) \mid k \in K(S_1) \cup K(S_2)\}$$

With these definitions, token-level reduction can be written as:

基于这些定义，标记级归约可表示为：

$$r_{\text{tok}}(v_1, v_2) = \begin{cases} \max(v_1, v_2) & \text{if } v_1 \neq \perp \wedge v_2 \neq \perp \\ v_1 & \text{if } v_1 \neq \perp \wedge v_2 = \perp \\ v_2 & \text{otherwise} \end{cases} \quad (6)$$

Defining the document-level reduction is slightly more complex as it involves incorporating the corresponding missing similarity estimates m . After token-level reduction each of the `query_maxlen` strides $S_1, \dots, S_{\text{query_maxlen}}$ covers scores for a single query token q_i . We set $S_{i,i} = S_i$ and define:

定义文档级归约稍显复杂，因其需要整合相应的缺失相似度估计值 m 。在词元级归约后，每个 `query_maxlen` 跨度 $S_1, \dots, S_{\text{query_maxlen}}$ 包含单个查询词元 q_i 的分数。我们设定 $S_{i,i} = S_i$ 并定义：

$$S_{i,j} = \text{reduce}(r_{\text{doc},(i,k,j)}, S_{i,k}, S_{k+1,j}) \quad (7)$$

for any choice of $i \leq k < j$, wherein $r_{\text{doc},(i,k,j)}$ merges two successive, non-overlapping strides $S_{i,k}$ and $S_{k+1,j}$. The resulting stride, $S_{i,j}$, now covers scores for query tokens $q_i, \dots, q_j$. Defining $r_{\text{doc},(i,k,j)}$ is relatively straightforward:

对于任意 $i \leq k < j$ 的选择，其中 $r_{\text{doc},(i,k,j)}$ 合并两个连续非重叠的跨度 $S_{i,k}$ 和 $S_{k+1,j}$ 。生成的跨度 $S_{i,j}$ 现在覆盖查询词元 $q_i, \dots, q_j$ 的分数。定义 $r_{\text{doc},(i,k,j)}$ 相对简单：

$$r_{\text{doc},(i,k,j)}(v_1, v_2) = \begin{cases} v_1 + v_2 & \text{if } v_1 \neq \perp \wedge v_2 \neq \perp \\ v_1 + \left(\sum_{t=k+1}^j m_t \right) & \text{if } v_1 \neq \perp \wedge v_2 = \perp \\ \left(\sum_{t=i}^k m_t \right) + v_2 & \text{otherwise} \end{cases} \quad (8)$$

It is easy to verify that $S_{i,j}$ is well-defined, i.e., independent of the choice of k . The result of document-level reduction is $S_{1, \text{query_maxlen}}$ and can be obtained by recursively applying Equation (7) to strides of increasing size.

容易验证 $S_{i,j}$ 是良定义的，即不依赖于 k 的选择。文档级归约的结果是 $S_{1, \text{query_maxlen}}$ ，可通过递归应用公式 (7) 到递增大小的跨度获得。

WARP's two-stage reduction process, along with the final sorting step, is illustrated in Figure 6. In the token-level reduction stage, strides are merged by selecting the maximum value for matching keys. In the document-level reduction stage, values for matching keys are summed, with missing values being substituted by the corresponding missing similarity estimates.

WARP 的两阶段归约流程及最终排序步骤如图 6 所示。词元级归约阶段通过选择匹配键的最大值合并跨度，文档级归约阶段则对匹配键的值求和，缺失值由相应缺失相似度估计值替代。

In our implementation, we conceptually construct a binary tree of the required merges and alternate between two scratch buffers to avoid additional memory allocations. We realize Equation (8) using a prefix sum, which eliminates the need to compute the sum explicitly.

本实现中，我们概念化构建所需合并的二叉树，并交替使用两个暂存缓冲区以避免额外内存分配。通过前缀和实现公式 (8)，无需显式计算总和。

4.6 Hyperparameters

4.6 超参数

In this section, we aim to analyze the effects of WARP’s three primary hyperparameters, namely:

本节旨在分析 WARP 三个主要超参数的影响:

- n_{probe} - the #clusters to decompress per query token
- n_{probe} - 每个查询词元解压缩的聚类数量
- t' - the threshold on the cluster size used for WARPSELECT
- t' - WARPSELECT 算法使用的聚类大小阈值
- b - the number of bits per dimension of a residual vector
- b - 残差向量每个维度的比特数

To study the effects of n_{probe} and t' , we analyze the normalized Recall @100¹⁰ as a function of t' for $n_{\text{probe}} \in \{1, 2, 4, 8, 16, 32, 64\}$ across four development datasets of increasing size: BEIR NFCorpus, BEIR Quora, LoTTE Lifestyle, and LoTTE Pooled. For further details on the datasets, please refer to Table 1. Figure 7 visualizes the results of our analysis. We observe a consistent pattern across all evaluated datasets, namely substantial improvements as n_{probe} increases from 1 to 16 (i.e., 1, 2, 4, 8, 16), followed by only marginal gains in Recall@100 beyond that. A notable exception is BEIR NFCorpus, where we still observe significant improvement when increasing from $n_{\text{probe}} = 16$ to $n_{\text{probe}} = 32$. We hypothesize that this is due to the small number of embeddings per cluster in NF-Corpus, limiting the number of scores available for aggregation. Consequently, we conclude that setting $n_{\text{probe}} = 32$ strikes a good balance between end-to-end latency and retrieval quality.

为研究 n_{probe} 和 t' 的影响，我们分析了四个规模递增的开发数据集 (BEIR NFCorpus、BEIR Quora、LoTTE Lifestyle 和 LoTTE Pooled) 上归一化召回率 @100¹⁰ 随 t' 变化的情况。数据集详情见表 1。图 7 展示了分析结果，所有数据集均呈现一致规律: 当 n_{probe} 从 1 增至 16 时召回率显著提升，超过该范围后 Recall@100 仅微弱增长。值得注意的是 BEIR NFCorpus 在 $n_{\text{probe}} = 16$ 到 $n_{\text{probe}} = 32$ 区间仍保持明显增益，我们推测这是由于该数据集每个聚类的嵌入向量较少，限制了可聚合的分数数量。因此得出结论: $n_{\text{probe}} = 32$ 能在端到端延迟与检索质量间实现最佳平衡。

⁹ PLAID cannot adopt this approach directly, as it normalizes the vectors after decompression. Empirically, we find that that this normalization step has limited effect on the final embeddings as the residuals are already normalized prior to quantization.

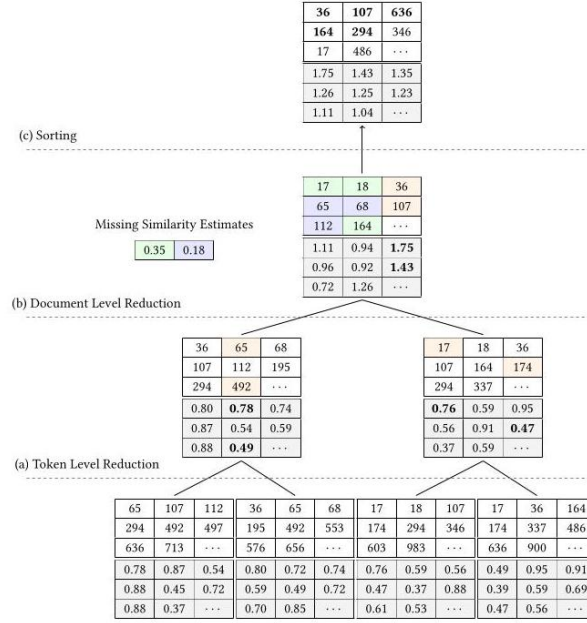


Figure 6: WARP’s scoring phase: (a) In token-level reduction, strides are max-reduced. (b) In document-level reduction, values are summed, accounting for missing similarity estimates. (c) Scores are sorted, yielding the top- k results.

图 6: WARP 评分阶段: (a) 令牌级归约采用最大值缩减步幅 (b) 文档级归约对相似性估计缺失值进行求和 (c) 排序后输出前 k 个结果

In general, we find that WARP is highly robust to variations in t' . However, smaller datasets, such as NFCorpus, appear to benefit from a smaller t' , while larger datasets perform better with a larger t' . Empirically, we find that setting t' proportional to the square root of the dataset size consistently yields strong results across all datasets. Moreover, increasing t' beyond a certain point no longer improves Recall, leading us to bound t' by a maximum value, $t'_{\max}$.

总体而言, WARP 对 t' 变化具有强鲁棒性。但较小数据集 (如 NFCorpus) 适合较小 t' , 而较大数据集需要更大 t' 。实证发现将 t' 设为数据集大小的平方根比例时效果最佳。当 t' 超过阈值后召回率不再提升, 故将其上限设为 $t'_{\max}$ 。

Next, we aim to quantify the effect of b on the retrieval quality of WARP, as shown in Figure 8. To do this, we compute the $n\text{Recall}@k$ for $n_{\text{probe}} = 32$ and $k \in \{10, 100\}$ using two datasets: LoTTE Science and LoTTE Pooled. Our results show a significant improvement in retrieval performance when increasing b from 2 to 4, particularly for

⁹ PLAID 无法直接采用此方法, 因其在解压后才对向量归一化。实证表明该归一化步骤对最终嵌入影响有限, 因残差在量化前已完成归一化处理。

¹⁰ The normalized Recall@k is calculated by dividing Recall@k by the dataset’s maximum, effectively scaling values between 0 and 1 to ensure comparability across datasets.

¹⁰ 归一化 Recall@k 通过将 Recall@k 除以数据集最大值计算, 将数值缩放至 0 到 1 区间以确保跨数据集可比性。

smaller values of k . For larger values of k , the difference in performance diminishes, particularly for the LoTTE Pooled dataset.

接下来我们量化 b 对 WARP 检索质量的影响 (图 8)。使用 LoTTE Science 和 LoTTE Pooled 数据集计算 $n_{\text{probe}} = 32$ 和 $k \in \{10, 100\}$ 的 $\text{nRecall}@k$ 。结果显示当 b 从 2 增至 4 时检索性能显著提升 (尤其在 k 较小时)。对于较大 k 值, 性能差异逐渐缩小 (LoTTE Pooled 数据集尤为明显)。

5 Evaluation

5 评估

We now evaluate WARP on six datasets from BEIR [18] and six datasets from LoTTE [17] listed in Table 1. We use servers with 28 Intel Xeon Gold 6132 @ 2.6 GHz CPU cores¹¹ and 500GB RAM. The servers have two NUMA sockets with roughly 92ns intra-socket memory latency, 142 ns inter-socket memory latency, 72 GBps intra-socket memory bandwidth, and 33 GBps inter-socket memory bandwidth.

我们在表 1 所列的 6 个 BEIR[18] 数据集和 6 个 LoTTE[17] 数据集上评估 WARP。实验服务器配置:28 核 Intel 至强金牌 6132@2.6GHz CPU¹¹, 500GB 内存, 含两个 NUMA 插槽 (插槽内内存延迟约 92ns 纳秒, 插槽间 142 纳秒; 插槽内内存带宽 72GB/s, 插槽间 33GB/s)。

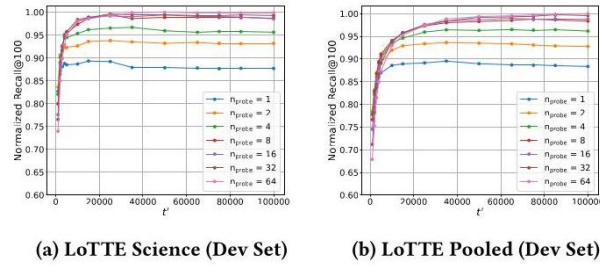


Figure 7: $\text{nRecall}@100$ as a function of t' and n_{probe}

图 7: $\text{nRecall}@100$ 随 t' 和 n_{probe} 变化曲线

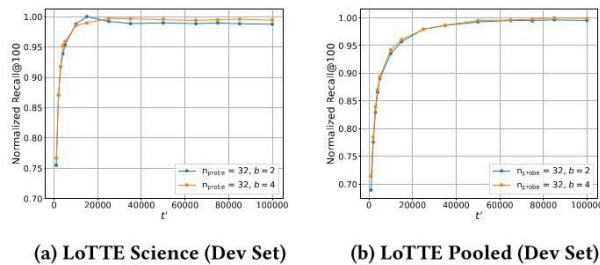


Figure 8: $\text{nRecall}@100$ as a function of t' and b

图 8: $\text{nRecall}@100$ 随 t' 和 b 变化曲线

Dataset		Dev		Test	
		#Queries	#Passages	#Queries	#Passages
BeIR [18]	NFCORPUS	324	3.6K	323	3.6K
	SciFact	-	-	300	5.2K
	SCIDOCS	-	-	1,000	25.7K
	Quora	5,000	522.9 K	10,000	522.9K
	FiQA-2018	500	57.6K	648	57.6K
	Touché-2020	-	-	49	382.5 K
LoTTE [17]	Lifestyle	417	268.9 K	661	119.5K
	Recreation	563	263.0K	924	167.0K
	Writing	497	277.1K	1,071	200.0K
	Technology	916	1.3M	596	638.5K
	Science	538	343.6K	617	1.7M
	Pooled	2,931	2.4M	3,869	2.8M

数据集		开发集		测试集	
		查询数量	段落数量	查询数量	段落数量
BeIR [18]	NFCORPUS	324	3.6K	323	3.6K
	SciFact	-	-	300	5.2K
	SCIDOCS	-	-	1,000	25.7K
	Quora	5,000	522.9 K	10,000	522.9K
	FiQA-2018	500	57.6K	648	57.6K
	Touché-2020	-	-	49	382.5 K
LoTTE [17]	生活方式	417	268.9 K	661	119.5K
	娱乐休闲	563	263.0K	924	167.0K
	写作	497	277.1K	1,071	200.0K
	技术	916	1.3M	596	638.5K
	科学	538	343.6K	617	1.7M
	汇总	2,931	2.4M	3,869	2.8M

Table 1: Datasets used for evaluating XTR_{base} /WARP performance. The evaluation includes 6 datasets from BEIR [18] and 6 from LoTTE [17].

表 1: 用于评估 XTR_{base} /WARP 性能的数据集。评估包含来自 BEIR[18] 的 6 个数据集和 LoTTE[17] 的 6 个数据集。

When measuring latency for end-to-end results, we compute the average latency of all queries Table 1 and report the minimum average latency across three trials. For other results, we describe the specific measurement procedure in the relevant section. We measure latency on an otherwise idle machine. As XTR ’s token retrieval stage does not benefit from GPU acceleration due to its use of ScaNN [5], specifically designed for single-threaded ¹² use on x86 processors with AVX2 support. Therefore, we perform CPU-only measurements and restrict the usage to a single thread unless otherwise stated.

测量端到端结果延迟时，我们计算表 1 中所有查询的平均延迟，并报告三次试验中的最小平均延迟。其他结果的测量方法将在相应章节详述。所有延迟测试均在空闲机器上进行。由于 XTR 的令牌检索阶段采用专为 x86 处理器 (支持 AVX2 指令集) 单线程¹² 设计的 ScaNN[5] 算法，无法受益于 GPU 加速，因此除非特别说明，我们仅进行 CPU 单线程测试。

	Lifestyle	Recreation	Writing	Technology	Science	Pooled	Avg.
BM25	63.8	56.5	60.3	41.8	32.7	48.3	50.6
ColBERT	80.2	68.5	74.7	61.9	53.6	67.3	67.7
GTR _{base}	82.0	65.7	74.1	58.1	49.8	65.0	65.8
XTR/ScaNN	83.5 (333.6)	69.6 (400.2)	78.0 (378.0)	63.9 (742.5)	55.3 (1827.6)	68.4 (2156.3)	69.8
WARP	83.5 (73.1)	69.5 (72.4)	78.6 (73.6)	64.6 (96.4)	56.1 (156.4)	69.3 (171.3)	70.3
Splade _{v2} * $\diamond$	82.3	69.0	77.1	62.4	55.4	68.9	69.2
ColBERT _{v2} ^{$\pm$} $\diamond$	84.7	72.3	80.1	66.1	56.7	71.6	71.9
GTR _{xxl}	87.4	78.0	83.9	69.5	60.0	76.0	75.8
XTR _{xxl}	89.1	79.3	83.3	73.7	60.8	77.3	77.3

	生活方式	休闲娱乐	写作	科技	科学	汇总	平均
BM25 算法	63.8	56.5	60.3	41.8	32.7	48.3	50.6
ColBERT 模型	80.2	68.5	74.7	61.9	53.6	67.3	67.7
GTR _{base}	82.0	65.7	74.1	58.1	49.8	65.0	65.8
XTR/ScaNN 技术	83.5 (333.6)	69.6 (400.2)	78.0 (378.0)	63.9 (742.5)	55.3 (1827.6)	68.4 (2156.3)	69.8
WARP 方法	83.5 (73.1)	69.5 (72.4)	78.6 (73.6)	64.6 (96.4)	56.1 (156.4)	69.3 (171.3)	70.3
Splade _{v2} * $\diamond$	82.3	69.0	77.1	62.4	55.4	68.9	69.2
ColBERT _{v2} ^{$\pm$} $\diamond$	84.7	72.3	80.1	66.1	56.7	71.6	71.9
GTR _{xxl}	87.4	78.0	83.9	69.5	60.0	76.0	75.8
XTR _{xxl}	89.1	79.3	83.3	73.7	60.8	77.3	77.3

-: cross-encoder distillation $\diamond$: model-based hard negatives

-: 交叉编码器蒸馏 (cross-encoder distillation) $\diamond$: 基于模型的困难负样本 (model-based hard negatives)

Table 2: Success@5 on LoTTE. Numbers in parentheses show average latency (milliseconds), with the final column displaying the average score across the datasets. Both XTR/ScaNN and WARP use the model XTR_{base}. XTR/ScaNN uses $k' = 40000$ and WARP uses $n_{\text{probe}} = 32$.

表 2: LoTTE 数据集上的 Success@5 指标。括号内数值表示平均延迟 (毫秒)，最后一列显示各数据集平均得分。XTR/ScaNN 和 WARP 均采用 XTR 模型_{base}，其中 XTR/ScaNN 使用 $k' = 40000$ ，WARP 使用 $n_{\text{probe}} = 32$ 。

¹¹ Each core has 2 threads for a total of 56 threads.

¹¹ 每个核心具有 2 线程，共计 56 线程。

	NFCorpus	SciFact	SCIDOCS	FiQA-2018	Touché-2020	Quora	Avg.
BM25	25.0	90.8	35.6	53.9	53.8	97.3	59.4
ColBERT	25.4	87.8	34.4	60.3	43.9	98.9	58.5
GTR _{base}	27.5	87.2	34.0	67.0	44.3	99.6	59.9
T5-ColBERT _{base}	27.6	91.3	34.2	63.0	49.9	97.9	60.7
XTR/ScaNN	28.0 (158.1)	90.9 (309.7)	34.3 (297.3)	62.0 (338.2)	50.3 (560.2)	98.9 (411.2)	60.7
WARP	27.9 (58.0)	92.8 (64.3)	36.8 (66.1)	62.3 (70.7)	51.5 (94.8)	99.0 (67.6)	61.7
GTR _{xxl}	30.0	90.0	36.6	78.0	46.6	99.7	63.5
T5-ColBERT _{xxl}	29.0	94.6	38.5	72.5	50.1	99.1	64.0
XTR _{xxl}	30.7	95.0	39.4	73.0	52.7	99.3	65.0

	NFCorpus	SciFact(科学事实库)	SCIDOCS(科学文献库)	FiQA-2018(金融问答数据集)	Touché-2020(论辩检索评测)	Quora(知乎海外版)	平均值
BM25 排序算法	25.0	90.8	35.6	53.9	53.8	97.3	59.4
ColBERT 模型	25.4	87.8	34.4	60.3	43.9	98.9	58.5
GTR _{base}	27.5	87.2	34.0	67.0	44.3	99.6	59.9
T5-ColBERT 混合模型 _{base}	27.6	91.3	34.2	63.0	49.9	97.9	60.7
XTR/ScaNN 检索系统	28.0 (158.1)	90.9 (309.7)	34.3 (297.3)	62.0 (338.2)	50.3 (560.2)	98.9 (411.2)	60.7
WARP 排序算法	27.9 (58.0)	92.8 (64.3)	36.8 (66.1)	62.3 (70.7)	51.5 (94.8)	99.0 (67.6)	61.7
GTR _{xxl}	30.0	90.0	36.6	78.0	46.6	99.7	63.5
T5-ColBERT 混合模型 _{xxl}	29.0	94.6	38.5	72.5	50.1	99.1	64.0
XTR _{xxl}	30.7	95.0	39.4	73.0	52.7	99.3	65.0

Table 3: Recall@100 on BEIR. Numbers in parentheses show average latency (milliseconds), with the final column displaying the average score across the datasets.

表 3: BEIR 数据集上的 Recall@100 指标。括号内数值表示平均延迟 (毫秒), 最后一列展示各数据集的平均得分。

5.1 End-to-End Results

5.1 端到端实验结果

Table 2 presents results on LoTTE. Our WARP method over the XTR model outperforms the optimized XTR_{base}/ScaNN implementation in terms of Success@5, while significantly reducing end-to-end latency, with speedups ranging from 4.6x on LoTTE Lifestyle to 12.8x on LoTTE Pooled. We observe a similar trend with the evaluation of nDCG@10 on the six BEIR [18] datasets, as shown in ???. XTR_{base}/WARP achieves speedups of 2.7x – 6x over XTR_{base}/ScaNN with a slight gain in nDCG@10. Likewise, we find improvements of Recall@100 on BEIR with substantial gains in end-to-end latency, but we omit them for the sake of space.

表 2 展示了 LoTTE 数据集上的结果。我们的 WARP 方法在 XTR 模型上实现了优于优化版 XTR_{base}/ScaNN 的 Success@5 表现, 同时显著降低端到端延迟, 在 LoTTE Lifestyle 数据集上加速达 4.6x, 在 LoTTE Pooled 数据集上加速达 12.8x。如??? 所示, 我们在六个 BEIR[18] 数据集上的 nDCG@10 评估也观察到类似趋势。XTR_{base}/WARP 在 nDCG@10 指标略有提升的同时, 相比 XTR_{base}/ScaNN 实现了 2.7x – 6x 的加速比。同样地, 我们在 BEIR 数据集上的 Recall@100 指标也观察到端到端延迟的大幅改善, 但限于篇幅未予展示。

5.2 Scalability

5.2 可扩展性分析

We now assess WARP’s scalability in relation to both dataset size and the degree of parallelism. To study the effect of the dataset size on WARP’s performance, we evaluate its latency across development datasets of varying sizes: BEIR NFCorpus, BEIR Quora, LoTTE Science, LoTTE Technology, and LoTTE Pooled (Table 1). Figure 9a plots the recorded latency of different configurations versus the size of the dataset, measured in the number of document token embeddings. Our results confirm, like in PLAID, that WARP’s latency scales in general with the square root of the dataset size—this is intuitive, as the number of clusters is by design proportional to the square root of the dataset size. Figure 9b illustrates WARP’s performance on the LoTTE Pooled development set, showing how the number of CPU threads impacts performance for different values of n_{probe} . Our results indicate that WARP effectively parallelizes across multiple threads, achieving a speedup of 3.1x for $n_{\text{probe}} = 32$ with 16 threads.

本节评估 WARP 在数据集规模和并行度两个维度上的可扩展性。为研究数据集规模对性能的影响，我们在不同规模的开发数据集 (BEIR NFCorpus、BEIR Quora、LoTTE Science、LoTTE Technology 和 LoTTE Pooled，见表 1) 上测量延迟。图 9a 绘制了不同配置下延迟随数据集规模 (以文档词嵌入数量计) 的变化曲线。与 PLAID 类似，实验结果证实 WARP 延迟基本与数据集规模的平方根成正比——这与设计原理一致，因为聚类数量默认与数据集规模的平方根成正比。图 9b 展示了 WARP 在 LoTTE Pooled 开发集上的表现，呈现了 n_{probe} 取值不同时 CPU 线程数对性能的影响。结果表明 WARP 能有效实现多线程并行，在 16 线程时对 $n_{\text{probe}} = 32$ 实现了 3.1x 的加速比。

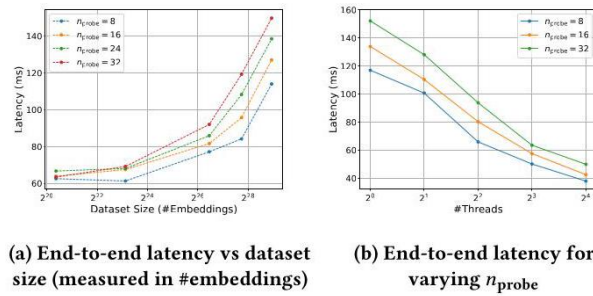


Figure 9: WARP’s scaling behaviour with respect to dataset size and the number of available CPU threads

图 9: WARP 算法随数据集规模和 CPU 线程数变化的扩展特性

5.3 Memory footprint

5.3 内存占用分析

Dataset		#Tokens	XTR Index Size (GiB)				
			BruteForce	FAISS	ScaNN	WARP _(b=2)	WARP _(b=4)
BelR [18]	NFCORPUS	1.35M	0.65	0.06	0.18	0.06	0.10
	SciFact	1.87M	0.91	0.08	0.25	0.07	0.13
	SCIDOCS	6.27M	3.04	0.28	0.82	0.24	0.43
	Quora	9.12M	4.43	0.43	1.21	0.35	0.62
	FiQA-2018	10.23M	4.95	0.46	1.34	0.38	0.69
	Touché-2020	92.64M	45.01	4.28	12.22	3.40	6.16
LoTTE [17]	Lifestyle	23.71M	11.51	1.08	3.12	0.88	1.59
	Recreation	30.04M	14.59	1.38	3.96	1.11	2.01
	Writing	32.21M	15.64	1.48	4.25	1.19	2.15
	Technology	131.92M	64.12	6.13	17.44	4.83	8.77
	Science	442.15M	214.93	20.57	58.46	16.07	29.28
	Pooled	660.04M	320.88	30.74	87.30	23.88	43.59
Total	-	1.44B	700.66	66.98	190.52	52.48	95.51

数据集		# 词元数	XTR 索引大小 (GiB)				
			暴力搜索	FAISS	ScaNN	WARP _(b=2)	WARP _(b=4)
BelR [18]	NFCORPUS	1.35M	0.65	0.06	0.18	0.06	0.10
	SciFact	1.87M	0.91	0.08	0.25	0.07	0.13
	SCIDOCS	6.27M	3.04	0.28	0.82	0.24	0.43
	Quora	9.12M	4.43	0.43	1.21	0.35	0.62
	FiQA-2018	10.23M	4.95	0.46	1.34	0.38	0.69
	Touché-2020	92.64M	45.01	4.28	12.22	3.40	6.16
LoTTE [17]	生活方式	23.71M	11.51	1.08	3.12	0.88	1.59
	休闲娱乐	30.04M	14.59	1.38	3.96	1.11	2.01
	写作	32.21M	15.64	1.48	4.25	1.19	2.15
	技术	131.92M	64.12	6.13	17.44	4.83	8.77
	科学	442.15M	214.93	20.57	58.46	16.07	29.28
	汇总	660.04M	320.88	30.74	87.30	23.88	43.59
总计	-	1.44B	700.66	66.98	190.52	52.48	95.51

Table 4: Comparison of index sizes for the datasets. Note that PLAID’s usage is memory usage is effectively identical to WARP’s, only slightly larger.

表 4: 各数据集索引大小的对比。需注意 PLAID 的内存使用效率与 WARP 几乎相同，仅略高。

Thanks to the adoption of a ColBERTv2- and PLAID-like approach for compression, WARP’s advantage over XTR extends also to a reduction in index size for XTR-based methods, which decreases memory requirements and, thus, broadens deployment options. Table 4 compares index sizes across all evaluated test datasets. WARP_(b=4) demonstrates a substantially smaller index size compared to both the BruteForce and ScaNN variants, providing a 7.3x and 2x reduction in memory footprint, respectively. While indexes generated by the FAISS implementation are marginally smaller, this comes at the cost of substantially reduced quality and latency. Notably, WARP_(b=2) outperforms the FAISS implementation in terms of quality with an even smaller index size.¹³

由于采用了类似 ColBERTv2 和 PLAID 的压缩方法，WARP 相较于 XTR 的优势还体现在基于 XTR 方法的索引体积缩减上，这不仅降低了内存需求，还拓宽了部署选择范围。表 4 对比了所有评估测试数据集的索引大小。WARP_(b=4) 相较于 BruteForce 和 ScaNN 变体展现出显著更小的索引体积，内存占用分别减少了 7.3x 和 2x。虽然 FAISS 实现生成的索引略小，但这是以显著降低质量和延迟为代价的。值得注意的是，WARP_(b=2) 在质量上优于 FAISS 实现，同时索引体积更小。¹³

6 Conclusion

6 结论

We introduce WARP, a highly optimized engine for multi-vector retrieval based on ColBERTv2 PLAID and the XTR framework. WARP overcomes inefficiencies of existing engines by: (1) the acceleration of query encoding using specialized inference runtimes, reducing inference latency by 2x – 3x, (2) WARP_{SELECT}, which dynamically adjusts to dataset characteristics while simultaneously decreasing computational overhead, and (3) an optimized two-stage reduction via a dedicated C++ kernel combined with implicit decompression. These optimizations culminate in substantial performance gains, including a 41x speedup over XTR on LoTTE Pooled, reducing latency from above 6s to just 171ms in single-threaded execution, and a 3x reduction in latency compared to ColBERTv2/PLAID, without negatively impacting retrieval quality. Beyond single-threaded performance, WARP shows significant speedup with increased thread count, and its reduced memory footprint enables deployment on resource-constrained devices.

我们推出 WARP——一个基于 ColBERTv2 PLAID 和 XTR 框架的高度优化多向量检索引擎。WARP 通过以下方式克服现有引擎的低效问题：(1) 采用专用推理运行时加速查询编码，将推理延迟降低 2x – 3x；(2) SELECT 动态适配数据集特征，同时减少计算开销；(3) 通过专用 C++ 内核结合隐式解压缩实现优化的两阶段归约。这些优化带来显著性能提升：在 LoTTE Pooled 上比 XTR 提速 41x（单线程执行延迟从 6 秒以上降至 171 毫秒），相较 ColBERTv2/PLAID 延迟降低 3 倍，且不影响检索质量。除单线程性能外，WARP 在增加线程数时也展现显著加速，其降低的内存占用使其可部署于资源受限设备。

References

参考文献

- [1] Thibault Formal, Stéphane Clinchant, Hervé Déjean, and Carlos Lassance. 2024.
- [2] Luyu Gao, Zhuyun Dai, and Jamie Callan. 2021. Coil: revisit exact lexical match in information retrieval with contextualized inverted list. (2021). <https://arxiv.org/abs/2104.07186> arXiv: 2104.07186 [cs.IR].
- [3] Stanford Future Data Systems Research Group. 2024. colbert-ir/colbertv2.0. <https://huggingface.co/colbert-ir/colbertv2.0>.(2024).

¹² As of the recently released version 1.3.0, ScaNN supports multi-threaded search via the `search_batched_parallel` function.

¹² 最新发布的 1.3.0 版本中，ScaNN 通过 `search_batched_parallel` 函数支持多线程搜索。

- [4] Stanford Future Data Systems Research Group. 2024. ColBERTv2/PLAID (Code). <https://github.com/stanford-futuredata/ColBERT>.(2024).
- [5] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. (2020). <https://arxiv.org/abs/1908.10396> arXiv: 1908.10396 [cs.LG].
- [6] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33, 1, 117-128. DOI: 10.1109/T-PAMI.2010.57.
- [7] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, (Eds.) Association for Computational Linguistics, Online, (Nov. 2020), 6769-6781. DOI: 10.18653/v1/2020.emnlp-main.550.
- [8] Omar Khattab and Matei Zaharia. 2020. Colbert: efficient and effective passage search via contextualized late interaction over BERT. *CoRR*, abs/2004.12832. <https://arxiv.org/abs/2004.12832> arXiv: 2004.12832.
- [9] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftexhar Naim, Ming-Wei Chang, and Vincent Y. Zhao. 2024. google/xtr-base-en. <https://huggingface.co/google/xtr-base-en>. (2024).
- [10] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftexhar Naim, Ming-Wei Chang, and Vincent Y. Zhao. 2024. Rethinking the role of token retrieval in multi-vector retrieval. (2024). arXiv: 2304.01982 [cs.CL].
- [11] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftexhar Naim, Ming-Wei Chang, and Vincent Y. Zhao. 2024. XTR: Rethinking the Role of Token Retrieval in Multi-Vector Retrieval (Code). <https://github.com/google-deepmind/xtr>. (2024).
- [12] Minghan Li, Sheng-Chieh Lin, Barlas Oguz, Asish Ghoshal, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2022. Citadel: conditional token interaction via dynamic lexical routing for efficient and effective multi-vector retrieval. (2022). <https://arxiv.org/abs/2211.10411> arXiv: 2211.10411 [cs.IR].
- [13] Franco Maria Nardini, Cosimo Rulli, and Rossano Venturini. 2024. Efficient multi-vector dense retrieval using bit vectors. (2024). arXiv: 2404.02805 [cs. IR].
- ¹³ Additionally, WARP reduces memory requirements compared to PLAID as it no longer requires storing a mapping from document ID to centroids/embeddings.
- 需要存储从文档 ID 到质心/嵌入向量的映射关系。
- [14] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. Exploring the limits of transfer learning with a unified text-to-text transformer. (2023). <https://arxiv.org/abs/1910.10683> arXiv: 1910.10683 [cs.LG].
- [15] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3, 4, 333-389. DOI: 10.1561/15000000019.
- [16] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022. Plaid: an efficient engine for late interaction retrieval. (2022). arXiv: 2205.09707 [cs.IR].
- [17] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2021. Colbertv2: effective and efficient retrieval via lightweight late interaction. *CoRR*, abs/2112.01488. <https://arxiv.org/abs/2112.01488> arXiv: 2112.01488.
- [18] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: a heterogeneous benchmark for zero-shot evaluation of information retrieval models. (2021). <https://arxiv.org/abs/2104.08663>

arXiv: 2104.08663 [cs. IR].

[19] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval. (2020). <https://arxiv.org/abs/2007.00808> arXiv: 2007.00808 [cs.IR].

[20] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. Optimizing dense retrieval model training with hard negatives. (2021). <https://arxiv.org/abs/2104.08051> arXiv: 2104.08051 [cs. IR].

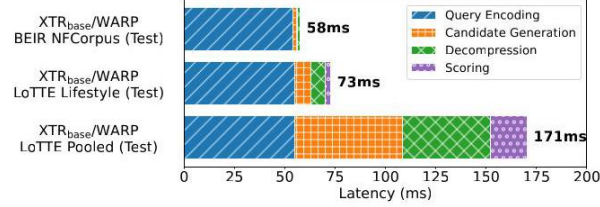


Figure 10: Breakdown of XTR_{base} /WARP's avg.

图 10: XTR_{base} /WARP 在

single-threaded latency for $n_{\text{probe}} = 32$ on the BEIR NFCorpus, LoTTE Lifestyle, and LoTTE Pooled datasets.

BEIR NFCorpus、LoTTE Lifestyle 和 LoTTE Pooled 数据集上的单线程延迟分解

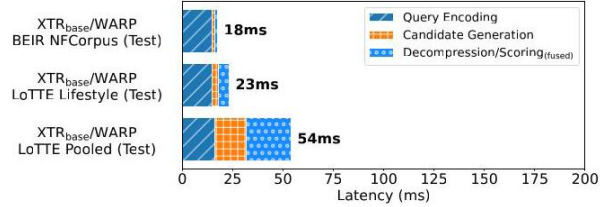


Figure 11: Breakdown of XTR_{base} /WARP's avg. latency for $n_{\text{probe}} = 32$ and $n_{\text{threads}} = 16$ on the BEIR NFCorpus, LoTTE Lifestyle, and LoTTE Pooled datasets

图 11: XTR_{base} /WARP 在 BEIR NFCorpus、LoTTE Lifestyle 和 LoTTE Pooled 数据集上针对 $n_{\text{probe}} = 32$ 与 $n_{\text{threads}} = 16$ 的平均延迟分解

A Additional Results

附录 A 补充结果

A.1 Latency Breakdowns

A.1 延迟分解

Next, we provide a more detailed breakdown of XTR_{base} /WARP’s performance on three datasets of varying sizes: BEIR NFCorpus [18], LoTTE Lifestyle [17], and LoTTE Pooled [17]. Figure 10 illustrates the latency breakdown across four key stages: query encoding, candidate generation, decompression, and scoring. For the smallest dataset, BEIR NFCorpus, the total latency is 58ms, with query encoding dominating the process. Moving to the larger LoTTE Lifestyle dataset, the total latency increases to 73 ms . Notably, on this dataset with over 100 K passages, WARP’s entire retrieval pipeline - comprising candidate generation, decompression, and scoring - constitutes only about 25% of the end-to-end latency, with the remaining time spent on query encoding. Even for the largest dataset, LoTTE Pooled, where the total latency reaches 171 ms , we observe that query encoding still consumes the majority of the processing time. While the other stages become more pronounced, query encoding remains the single most time-consuming stage of the retrieval process. Without the use of specialized inference run-times, query encoding accounts for approximately half of the execution time, thus presenting the primary bottleneck for end-to-end retrieval using WARP.

接下来我们详细分析 XTR_{base} /WARP 在三个不同规模数据集 (BEIR NFCorpus[18]、LoTTE Lifestyle[17] 和 LoTTE Pooled[17]) 上的性能表现。图 10 展示了四个关键阶段的延迟分解: 查询编码、候选生成、解压缩和评分。最小数据集 BEIR NFCorpus 总延迟为 58 毫秒, 其中查询编码占主导。较大规模的 LoTTE Lifestyle 数据集总延迟增至 73 ms 。值得注意的是, 在这个包含超过 100 K 段落的数据集上, WARP 的整个检索流程 (含候选生成、解压缩和评分) 仅占端到端延迟的 25%, 其余时间消耗在查询编码。即便对于最大的 LoTTE Pooled 数据集 (总延迟达 171 ms), 查询编码仍占处理时间的主要部分。虽然其他阶段耗时增加, 但查询编码始终是检索过程中最耗时的环节。若不使用专用推理运行时, 查询编码约占执行时间的一半, 成为 WARP 端到端检索的主要瓶颈。

A key advantage of WARP over the reference implementation is its ability to leverage multi-threading, thereby significantly improving performance. Figure 11 illustrates the end-to-end latency breakdown for WARP using 16 threads. The decompression and scoring stages are fused in multi-threaded contexts. WARP demonstrates great scalability, achieving substantial latency reduction across all stages. In the 16-thread configuration, it notably surpasses the GPU-based implementation of PLAID on the LoTTE Pooled dataset.

WARP 相较于参考实现的一个关键优势在于其能够利用多线程技术, 从而显著提升性能。图 11 展示了 WARP 使用 16 线程时的端到端延迟分解情况。在多线程环境下, 解压与评分阶段实现了融合处理。WARP 展现出优异的可扩展性, 在所有处理阶段都实现了显著的延迟降低。在 16 线程配置下, 其在 LoTTE Pooled 数据集上的表现明显优于基于 GPU 的 PLAID 实现方案。

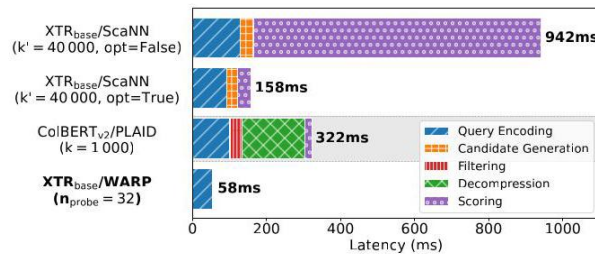


Figure 12: Latency breakdown of the unoptimized reference implementation, optimized variant, ColBERTv2/PLAID, and XTR_{base} /WARP on BEIR NFCorpus Test

图 12: BEIR NFCorpus 测试集上未优化参考实现、优化变体、ColBERTv2/PLAID 与 XTR_{base} /WARP 的延迟分解对比

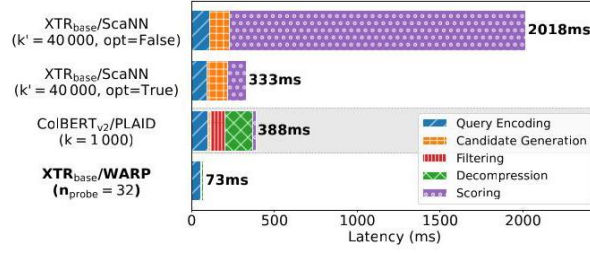


Figure 13: Latency breakdown of the unoptimized reference implementation, optimized variant, ColBERTv2/PLAID, and XTR_{base} /WARP on LoTTE Lifestyle Test

图 13: LoTTE Lifestyle 测试集上未优化参考实现、优化变体、ColBERTv2/PLAID 与 XTR_{base} /WARP 的延迟分解对比

A.2 Performance Comparisons

A.2 性能对比

Similar to Figure 1, we analyze the performance of WARP and contrast it with the performance of the baseline on the BEIR NFCorpus (Figure 12) and LoTTE Lifestyle (Figure 13) datasets. We find that WARP’s single-threaded end-to-end latency is dominated by query encoding on BEIR NFCorpus and LoTTE Lifestyle, whereas the baselines introduce significant overhead via their retrieval pipeline.

与图 1 类似，我们分析了 WARP 在 BEIR NFCorpus(图 12) 和 LoTTE Lifestyle(图 13) 数据集上的性能表现，并与基线进行对比。研究发现，在单线程端到端延迟方面，WARP 在 BEIR NFCorpus 和 LoTTE Lifestyle 上的主要耗时来自查询编码环节，而基线方法则因其检索流程引入了显著开销。

A.3 Evaluation of ColBERTv2/WARP

A.3 ColBERTv2/WARP 评估

To assess WARP’s ability to generalize beyond the XTR model, we conduct experiments using ColBERTv2 in place of XTR_{base} for query encoding. The results, presented in Table 5, show that WARP performs competitively with PLAID, despite not being specifically designed for retrieval with ColBERTv2. This suggests that WARP’s approach may generalize effectively to retrieval models other than XTR. We leave more detailed analysis for future work.

为评估 WARP 在 XTR 模型之外的泛化能力，我们采用 ColBERTv2 替代 XTR_{base} 进行查询编码实验。表 5 结果显示，尽管 WARP 并非专为 ColBERTv2 检索设计，其性能仍与 PLAID 相当，这表明 WARP 的方法可能有效适用于 XTR 之外的其他检索模型。更深入的分析将留待后续研究。

	NFCorpus	SciFact	SCIDOCS	FiQA-2018	Touché-2020	Quora	Avg.
ColBERT _{v2} /PLAID (k= 10)	33.3	69.0	15.3	34.5	25.6	85.1	43.8
ColBERT _{v2} /PLAID (k= 100)	33.4	69.2	15.3	35.4	25.2	85.4	44.0
ColBERT _{v2} / PLAID (k = 1000)	33.5	69.2	15.3	35.5	25.6	85.5	44.1
ColBERT _{v2} /WARP ($n_{\text{probe}} = 32$)	34.6	70.6	16.2	33.6	26.4	84.5	44.3

	NFCorpus	SciFact(科学事实库)	SCIDOCS(科学文献库)	FiQA-2018(金融问答数据集)	Touché-2020(论辩检索评测)	Quora(知乎海外版)	平均值
ColBERT _{v2} /PLAID (k=10)	33.3	69.0	15.3	34.5	25.6	85.1	43.8
ColBERT _{v2} /PLAID (k=100)	33.4	69.2	15.3	35.4	25.2	85.4	44.0
ColBERT _{v2} / PLAID (k = 1000)	33.5	69.2	15.3	35.5	25.6	85.5	44.1
ColBERT _{v2} /WARP ($n_{\text{probe}} = 32$)	34.6	70.6	16.2	33.6	26.4	84.5	44.3

Table 5: ColBERT_{v2}/WARP nDCG@10 on BEIR. The last column shows the average over 6 BEIR datasets.

表 5: ColBERT_{v2}/WARP 在 BEIR 上的 nDCG@10 指标。最后一列显示 6 个 BEIR 数据集的平均值。

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

收稿日期:2007 年 2 月 20 日; 修回日期:2009 年 3 月 12 日; 录用日期:2009 年 6 月 5 日